Efficient PE Header Feature Extraction for Malware Analysis: Comparing Reduced and Extended Feature Sets

Fabiha Jalal, Sadia Tasnim Dhruba, Onamika Hossain
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Gazipur, Bangladesh
Email: fabihajalal@iut-dhaka.edu

Dr. Md Moniruzzaman
Supervisor, Assistant Professor
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Gazipur, Bangladesh

***Abstract*—Malware detection remains a critical challenge in cybersecurity as the volume and sophistication of malicious software continue to increase exponentially. Notably, prior studies on the APT1 dataset reported feature extraction times exceeding 19 minutes (1,157.81 seconds) for processing executable samples, making real-time detection impractical. This research presents an efficient feature extraction methodology for static malware analysis utilizing Portable Executable (PE) header files. We conduct a comprehensive comparative study between two distinct feature set approaches: a reduced feature set (14 features) and an extended feature set (32 features) extracted from PE file headers. Our experiments, conducted on a newly created and publicly available dataset of 1,150 malware executable samples collected from MalwareBazaar, demonstrate that the extended feature set achieves superior classification accuracy (94.78%) compared to the reduced feature set (93.91%) using Random Forest classification, with Decision Tree also achieving 93.48% accuracy on the extended set. Our proposed methodology reduces feature extraction time from approximately 19 minutes to approximately 215–290 seconds, representing a reduction of over 75%. The research evaluates six machine learning classifiers including Random Forest, K-Nearest Neighbors, Decision Tree, Support Vector Machine, Logistic Regression, and Gradient Boosting, providing insights into the trade-offs between feature dimensionality, extraction time, and classification performance. Additionally, we create and publish a novel malware dataset with pre-extracted PE header features for research purposes. Our findings contribute to the development of more efficient real-time malware detection systems by reducing feature extraction time.**

***Index Terms*—PE Header, Feature Extraction, Extraction Time, Machine Learning, Cybersecurity**

I. Introduction

A. Overview

The proliferation of malware poses a critical threat to computer systems worldwide, with over 1 billion malware instances detected in 2023 alone, averaging approximately 33 new malware samples per second [1]. The increasing frequency and sophistication of malware attacks necessitate the development of efficient and accurate detection methodologies. Malware analysis, the process of understanding the functionality and purpose of malicious software, plays a crucial role in identifying threats, assessing potential damage, and developing effective countermeasures.

Static malware analysis, which examines executable files without execution, offers several advantages over dynamic analysis, including safety, speed, and the ability to analyze malware that employs anti-analysis techniques [2]. By leveraging structural information contained within Portable Executable (PE) files, static analysis can efficiently identify malicious characteristics without the computational overhead and security risks associated with runtime analysis.

B. Research Objectives and Contributions

This research aims to develop an efficient feature extraction methodology using PE header files that minimizes extraction time while maintaining classification accuracy. To this end, we conduct a comparative analysis between a reduced (14 features) and an extended (32 features) feature set, and evaluate six machine learning classifiers for malware detection. Additionally, we create and publish a novel malware dataset of 1,150 samples with pre-extracted PE header features for research purposes, and demonstrate the trade-offs between feature dimensionality, extraction time, and detection performance.

The primary contributions of this work are as follows. First, we propose a novel and efficient feature extraction technique that reduces extraction time from approximately 19 minutes to under 5 minutes. Second, we provide a publicly available dataset of 1,150 malware executable files with extracted PE header features. Third, we present a comprehensive comparative analysis of two feature set approaches across six machine learning classifiers, providing empirical evidence on the relationship between feature quantity and classification performance.

II. Background and History

A. Malware Evolution

Malware, derived from “malicious software,” encompasses any program designed to damage, exploit, or gain unauthorized access to computer systems. The evolution of malware has progressed significantly since the emergence of early computer viruses in the 1980s. Modern malware categories include:

Viruses: Self-replicating programs that attach to legitimate files and propagate when infected files are executed.

Worms: Self-replicating malware that spreads through network connections without requiring host files.

Trojans: Malicious programs disguised as legitimate software that create backdoors, steal data, or enable unauthorized access.

Ransomware: Malware that encrypts victim files and demands payment for decryption keys. Notable examples include WannaCry (2017) and NotPetya (2017) [10].

Spyware: Software that covertly monitors user activities, captures keystrokes, and transmits sensitive information to unauthorized parties.

B. Malware Analysis Techniques

1) Static Analysis: Static analysis examines malware without execution by analyzing file metadata, extracting strings, and reading file headers [5]. This approach is safer, faster, and can identify malicious indicators that might be hidden during dynamic execution. Key components of static analysis include:

- File hash verification (MD5, SHA-256)
- PE header examination
- String extraction and analysis
- Disassembly and code pattern recognition

2) Dynamic Analysis: Dynamic analysis involves executing malware in controlled environments (sandboxes or virtual machines) to observe runtime behavior, including system call monitoring, network traffic analysis, file system modifications, and registry changes.

3) Hybrid Analysis: Hybrid analysis combines static and dynamic techniques to provide comprehensive malware characterization, leveraging the strengths of both approaches [8].

C. Portable Executable (PE) File Format

The PE file format is the standard executable format for Windows operating systems, containing structural information essential for operating system loaders [4]. The PE structure begins with the DOS Header (MZ Header), which contains the magic number (0x5A4D) identifying the file as a DOS-compatible executable, followed by the DOS Stub that provides legacy DOS compatibility. The PE File Header contains critical metadata including machine architecture type, number of sections, timestamp, optional header size, and characteristics flags. The Optional Header, despite its name, is mandatory for executable files and contains entry point address, image base address, section and file alignment, operating system version requirements, subsystem type, DLL characteristics, and stack/heap size specifications [6]. Finally, the Section Table describes the layout of executable sections (.text, .data, .rdata, .rsrc, etc.). The PE header structure provides valuable features for malware classification, as malicious executables often exhibit distinctive header characteristics that differ from benign software [7], [9].

III. Motivation

A. The Need for Efficient Feature Extraction

The exponential growth in malware variants demands analysis techniques that can process large volumes of samples efficiently. In 2021, ransomware attacks occurred approximately every 11 seconds, highlighting the critical need for rapid detection capabilities.

Previous research utilizing PE header features for malware detection, such as studies on the APT1 dataset [3], reported feature extraction times exceeding 19 minutes (1,157.81 seconds) for processing the entire dataset of executable samples. Such prolonged extraction times are impractical for real-time malware detection systems that must analyze thousands of samples daily.

B. Feature Selection Considerations

The selection of PE header features significantly impacts both extraction efficiency and classification accuracy. While comprehensive feature sets may capture more discriminative information, they also increase computational overhead. Conversely, minimal feature sets may sacrifice accuracy for speed.

This research investigates the optimal balance between feature comprehensiveness and extraction efficiency by comparing various numbers of features. We selected a reduced feature set of 14 attributes focusing on essential PE header fields that are most commonly referenced in prior malware detection studies, and an extended feature set of 32 attributes that incorporates all accessible FILE_HEADER and OPTIONAL_HEADER fields. These two sizes were chosen to represent a minimal, fast-extraction baseline versus a comprehensive, information-rich alternative. While other intermediate feature set sizes (e.g., 20 or 25 features) could also be explored, our goal was to establish the boundary conditions—the smallest practical set and the largest available set—to clearly quantify the trade-off between extraction time and classification accuracy.

Prior studies have demonstrated the effectiveness of PE header-based malware detection but have not adequately addressed the trade-off between feature quantity and extraction time, comparative analysis of different feature set sizes on the same dataset, or optimization of feature extraction for real-time detection applications. This research addresses these gaps through systematic experimentation and analysis.

IV. Working Mechanism

A. System Architecture

The proposed malware detection system comprises three primary phases: (1) Feature Extraction from PE files using the pefile library, (2) Machine Learning Classification using trained models, and (3) Detection Result output. Figure 1 illustrates this workflow.

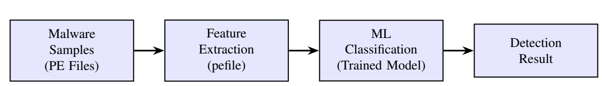


Fig. 1: System Architecture Overview

B. Dataset Preparation

1) Data Collection: Malware executable samples were collected from MalwareBazaar [11], a reputable malware sample repository. The dataset comprises 1,150 PE executable files representing various malware families.

2) Data Processing: The Python library was employed for PE header parsing. Files were validated to ensure proper PE format before feature extraction. Non-PE files (.rar, .elt, etc.) were excluded from the dataset.

C. Feature Extraction Methodology

1) Reduced Feature Set (14 Features): The reduced feature set extracts 14 attributes from PE headers, with Characteristics used as the target variable. From the FILE_HEADER, two features are extracted: Machine and NumberOfSections. From the OPTIONAL_HEADER, twelve features are extracted: AddressOfEntryPoint, ImageBase, SectionAlignment, FileAlignment, Subsystem, DllCharacteristics, MajorOperatingSystemVersion, MinorOperatingSystemVersion, MajorImageVersion, MinorImageVersion, SizeOfImage, and SizeOfHeaders.

2) Extended Feature Set (32 Features): The extended feature set extracts 32 attributes, with Characteristics used as the target variable. From the FILE_HEADER, six features are extracted: Machine, NumberOfSections, TimeDateStamp, PointerToSymbolTable, NumberOfSymbols, and SizeOfOptionalHeader. From the OPTIONAL_HEADER, twenty-six features are extracted: MajorLinkerVersion, MinorLinkerVersion, SizeOfCode, SizeOfInitializedData, SizeOfUninitializedData, AddressOfEntryPoint, BaseOfCode, ImageBase, SectionAlignment, FileAlignment, MajorOperatingSystemVersion, MinorOperatingSystemVersion, MajorImageVersion, MinorImageVersion, MajorSubsystemVersion, MinorSubsystemVersion, SizeOfHeaders, CheckSum, Subsystem, DllCharacteristics, SizeOfStackReserve, SizeOfStackCommit, SizeOfHeapReserve, SizeOfHeapCommit, LoaderFlags, and NumberOfRvaAndSizes.

D. Feature Extraction Procedure

The feature extraction process is implemented using Python with the library. Algorithm 1 presents the pseudocode for the extraction process.

Algorithm 1 PE Header Feature Extraction

Require: Directory path containing PE files
Ensure: CSV file with extracted features

```
for each file in directory do
    Parse PE file using pefile library
    if PE parsing successful then
        Extract FILE_HEADER features
        Extract OPTIONAL_HEADER features
        Append features to dataset
    else
        Log error and skip file
    end if
end for
Save dataset to CSV file
```

E. Machine Learning Classification

Six machine learning classifiers were employed for malware detection. Random Forest is an ensemble method using 100 decision trees with bootstrap aggregation. K-Nearest Neighbors (KNN) is an instance-based learning approach using Euclidean distance with $k = 5$. Decision Tree performs tree-based classification using information gain. Support Vector Machine (SVM) uses kernel-based classification with an RBF kernel. Logistic Regression performs linear classification with maximum likelihood estimation. Gradient Boosting is a sequential ensemble method with 50 estimators.

F. Experimental Setup

The dataset was split into 80% training (920 samples) and 20% testing (230 samples). Model validation was performed using 3-fold cross-validation, consistent across both feature sets. A random state of 42 was used for reproducibility. The evaluation metrics employed include Accuracy, Precision, Recall, and F1-Score.

V. Results

A. Feature Extraction Performance

Table I presents the feature extraction performance comparison between the two approaches.

TABLE I: Feature Extraction Performance Comparison

Metric	**Reduced (14)**	**Extended (32)**
Extraction Time	215.81 sec	290.12 sec
Processing Speed	5.33 files/sec	3.96 files/sec
Files Processed	1,150	1,150
Errors	0	0
CSV Columns	16	34

Comparison with Prior Work (APT1 Dataset):

- APT1 PE Header Extraction: 1,157.81 seconds (∼19 minutes)
- Our Extended Feature Set: 290.12 seconds (∼4.8 minutes)
- **Improvement: 75% reduction in extraction time**

B. Classification Results - Reduced Feature Set

Table II presents the classification results for the reduced feature set (14 features).

TABLE II: Classification Results - Reduced Feature Set (14 Features)

Classifier	Accuracy	F1	Precision	Recall
Random Forest	**93.91%**	0.8379	0.8858	0.8264
Decision Tree	91.74%	0.7329	0.7681	0.7302
Gradient Boosting	90.87%	0.5956	0.6037	0.6028
KNN	75.22%	0.4636	0.4969	0.4590
Logistic Regression	51.30%	0.1369	0.1309	0.1734
SVM	25.65%	0.0240	0.0151	0.0588

As shown in Table II, tree-based classifiers dominate performance with the reduced feature set. Random Forest achieves the highest accuracy of 93.91% with an F1-score of 0.8379 and the best precision of 0.8858, indicating that ensemble methods effectively leverage even a limited set of PE header features for malware classification. Decision Tree follows closely at 91.74% accuracy with an F1-score of 0.7329, while Gradient Boosting achieves 90.87% accuracy but with a notably lower F1-score of 0.5956, suggesting it struggles with certain malware classes despite reasonable overall accuracy. KNN shows moderate performance at 75.22%, indicating that distance-based methods have difficulty distinguishing malware families in the limited 14-dimensional feature space. Logistic Regression and SVM perform poorly at 51.30% and 25.65% respectively, which is likely attributable to the multi-class nature of the classification problem and the absence of feature scaling—both of which are critical for optimal performance of these linear classifiers.

C. Classification Results - Extended Feature Set

Table III presents the classification results for the extended feature set (32 features).

TABLE III: Classification Results - Extended Feature Set (32 Features)

Classifier	Accuracy	F1	Precision	Recall
Random Forest	**94.78%**	0.8632	0.9051	0.8399
Decision Tree	93.48%	0.7993	0.8167	0.7927
Gradient Boosting	93.04%	0.7209	0.7616	0.7133
KNN	80.00%	0.5085	0.5300	0.5053
Logistic Regression	41.30%	0.1098	0.1076	0.1368
SVM	25.65%	0.0240	0.0151	0.0588

With the extended feature set of 32 attributes, all tree-based classifiers show notable improvement over the reduced set. Random Forest achieves the best accuracy of 94.78% with an F1-score of 0.8632 and precision of 0.9051, confirming that additional PE header features provide more discriminative power for ensemble methods. Decision Tree improves to 93.48% accuracy with an F1-score of 0.7993, and Gradient Boosting reaches 93.04% with a substantially improved F1-score of 0.7209 compared to 0.5956 in the reduced set, indicating that the additional features help resolve previously ambiguous classifications. KNN benefits the most from the extended features, rising from 75.22% to 80.00%, a 4.78 percentage point improvement, which suggests that the additional features create more meaningful distance separations in the higher-dimensional feature space. Notably, Logistic Regression drops from 51.30% to 41.30%, likely due to multicollinearity among the additional correlated features, which degrades the performance of linear models. SVM remains unchanged at 25.65%, confirming its unsuitability for this multi-class problem without proper feature scaling and kernel optimization.

D. Comparative Analysis

Table IV presents the accuracy improvement achieved by the extended feature set across all six classifiers.

TABLE IV: Accuracy Comparison Between Feature Sets

Classifier	Reduced	Extended	Improvement
Random Forest	93.91%	94.78%	+0.87%
Decision Tree	91.74%	93.48%	+1.74%
Gradient Boosting	90.87%	93.04%	+2.17%
KNN	75.22%	80.00%	+4.78%
Logistic Regression	51.30%	41.30%	−10.00%
SVM	25.65%	25.65%	0.00%

Table IV reveals a clear pattern in how different classifier families respond to additional features. Tree-based and instance-based classifiers consistently benefit from the extended feature set, with improvements ranging from +0.87% for Random Forest to +4.78% for KNN. The relatively small improvement for Random Forest (+0.87%) suggests that it already captures most of the discriminative information with just 14 features, while KNN's substantial improvement (+4.78%) indicates that the additional features create more separable clusters in higher-dimensional space, enabling better distance-based classification. Gradient Boosting shows a +2.17% improvement, demonstrating that sequential ensemble methods also benefit from richer feature representations. In contrast, Logistic Regression suffers a 10% drop in accuracy with the extended set, highlighting the curse of dimensionality for linear models when features are correlated. SVM shows no change, remaining at 25.65% for both sets, which confirms that without proper feature scaling and hyperparameter optimization, kernel-based methods cannot effectively leverage PE header features for multi-class malware classification. These results suggest that the choice of feature set should be guided by the intended classifier—tree-based methods can work well with either set, while linear classifiers require careful feature engineering.

E. Feature Importance Analysis

Table V presents the top 10 most important features for malware classification using Random Forest on the extended feature set.

TABLE V: Top 10 Feature Importance (Random Forest)

Rank	Feature	Importance
1	AddressOfEntryPoint	0.095
2	MajorLinkerVersion	0.092
3	TimeDateStamp	0.084
4	SizeOfCode	0.068
5	DllCharacteristics	0.065
6	ImageBase	0.057
7	SizeOfInitializedData	0.057
8	SizeOfOptionalHeader	0.050
9	Machine	0.050
10	NumberOfSections	0.039

F. Performance Visualization - Reduced Feature Set

Figure 2 shows the classifier performance comparison for the reduced feature set.

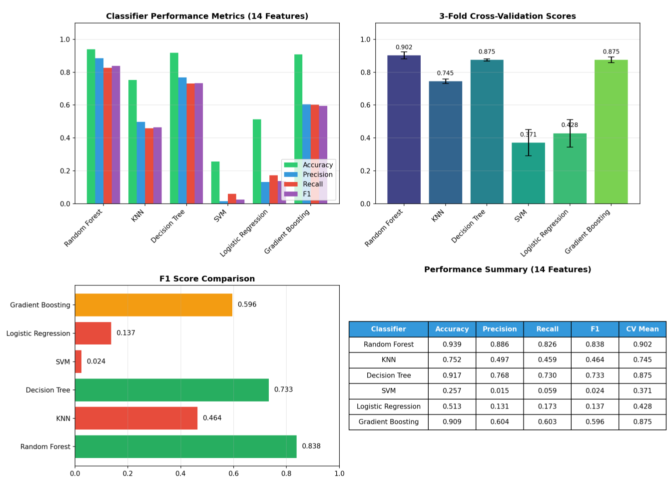


Fig. 2: Classifier Performance Comparison - Reduced Feature Set (14 Features)

Figure 2 displays accuracy, precision, recall, and F1-score for all six classifiers using the reduced feature set. Random Forest dominates across all four metrics, while the sharp drop-off from Gradient Boosting to KNN illustrates the performance boundary between tree-based and non-tree-based methods on this feature set.

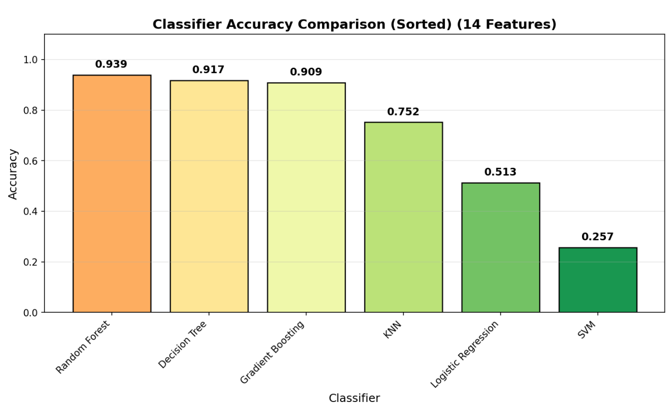


Fig. 3: Accuracy Comparison - Reduced Feature Set (14 Features)

Figure 3 presents the accuracy values sorted in descending order. The top three classifiers (Random Forest at 93.91%, Decision Tree at 91.74%, and Gradient Boosting at 90.87%) are clearly separated from the lower-performing classifiers, with KNN at 75.22% forming a middle tier and Logistic Regression (51.30%) and SVM (25.65%) performing near or below random chance for this multi-class problem.

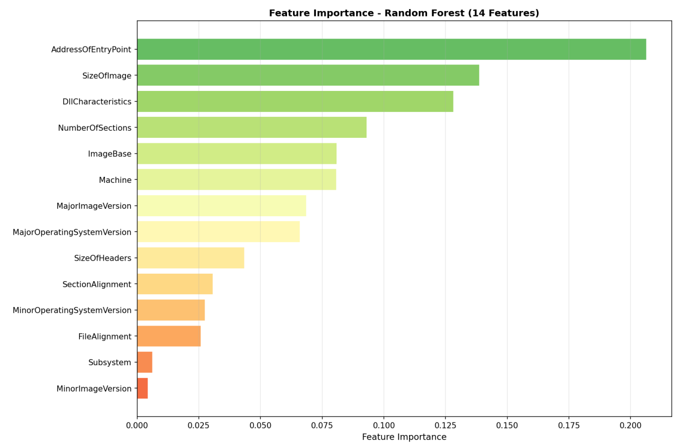


Fig. 4: Feature Importance - Reduced Feature Set (14 Features)

Figure 4 shows the relative importance of each feature in the reduced set as determined by the Random Forest classifier. AddressOfEntryPoint emerges as the most important feature, followed by SizeOfImage and DllCharacteristics. The distribution indicates that a few features contribute disproportionately to classification decisions, suggesting potential for further feature reduction without major accuracy loss.

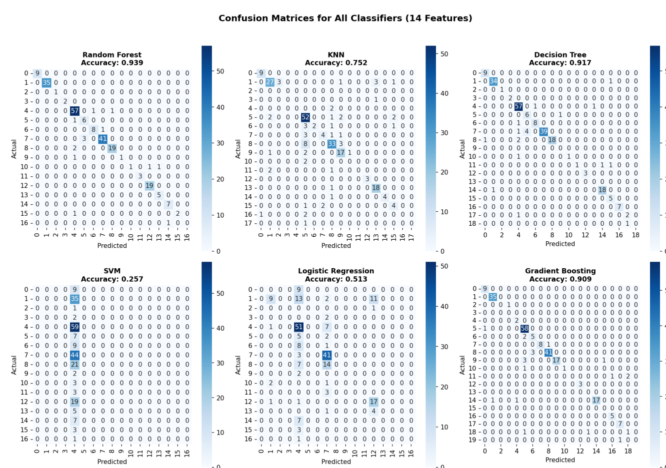


Fig. 5: Confusion Matrices - Reduced Feature Set (14 Features)

Figure 5 presents the confusion matrices for all six classifiers. Random Forest and Decision Tree show strong diagonal dominance indicating correct classifications across most malware families, while SVM and Logistic Regression exhibit scattered predictions with many off-diagonal entries, confirming their inability to distinguish between malware classes effectively with this feature set.

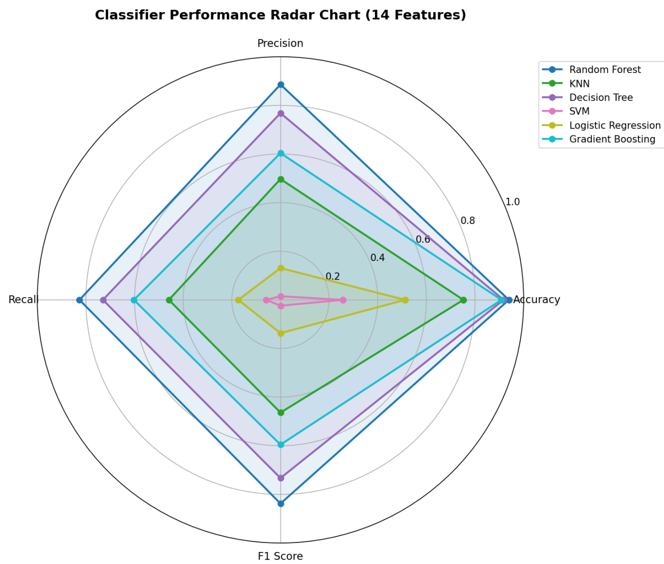
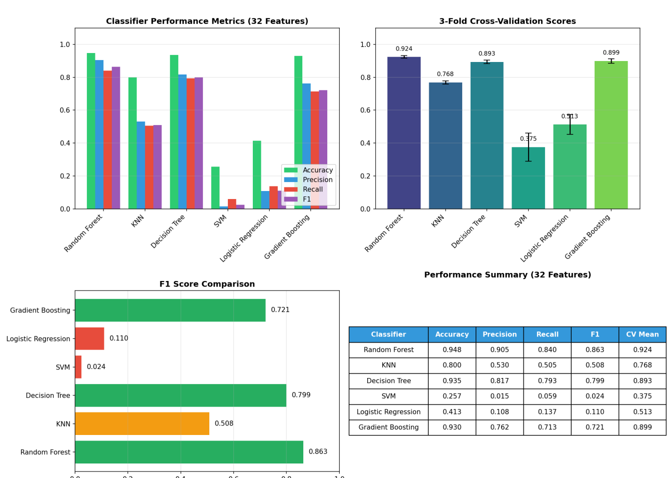


Fig. 6: Performance Radar Chart - Reduced Feature Set (14 Features)

Figure 6 provides a radar chart visualizing precision, recall, accuracy, and F1-score simultaneously for all classifiers. Random Forest covers the largest area, indicating balanced and superior performance across all metrics. The collapsed shapes of SVM and Logistic Regression near the center confirm their poor multi-metric performance.

G. Performance Visualization - Extended Feature Set

Figure 7 shows the classifier performance comparison for the extended feature set.



Classifier	Accuracy	Precision	Recall	F1	CV Mean
Random Forest	0.948	0.905	0.840	0.863	0.924
KNN	0.800	0.530	0.505	0.508	0.768
Decision Tree	0.935	0.817	0.793	0.799	0.893
SVM	0.257	0.015	0.059	0.024	0.375
Logistic Regression	0.413	0.108	0.137	0.110	0.513
Gradient Boosting	0.930	0.762	0.713	0.721	0.899

Fig. 7: Classifier Performance Comparison - Extended Feature Set (32 Features)

Figure 7 displays the performance metrics for all classifiers using the extended 32-feature set. Compared to the reduced set, the bars for Random Forest, Decision Tree, Gradient Boosting, and KNN are visibly taller across all metrics, confirming the benefit of additional features for these classifiers. The gap between Gradient Boosting and KNN has narrowed, suggesting that the extended features help instance-based methods more proportionally.

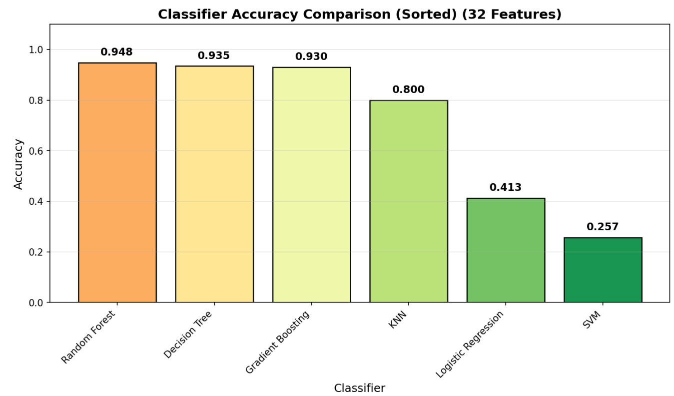


Fig. 8: Accuracy Comparison - Extended Feature Set (32 Features)

Figure 8 shows the sorted accuracy comparison for the extended feature set. Random Forest leads at 94.78%, followed closely by Decision Tree (93.48%) and Gradient Boosting (93.04%). KNN's improvement to 80.00% is particularly notable. The persistent poor performance of Logistic Regression (41.30%) and SVM (25.65%) highlights that additional features do not compensate for fundamental classifier limitations in this multi-class setting.

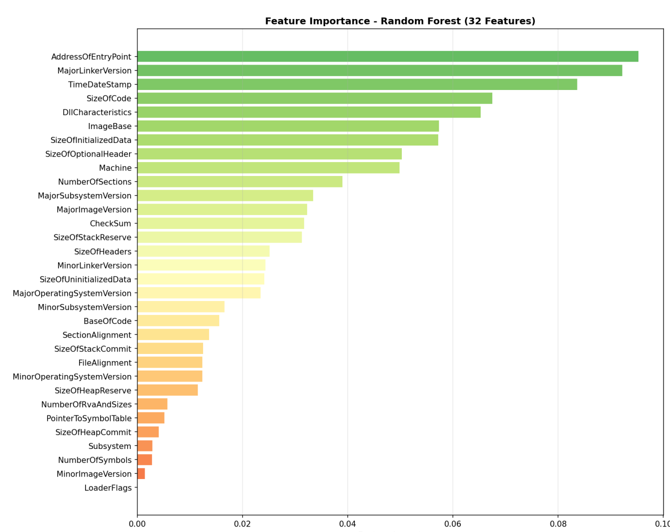


Fig. 9: Feature Importance - Extended Feature Set (32 Features)

Figure 9 illustrates the importance ranking of all 32 features as determined by Random Forest. The distribution is more gradual compared to the reduced set, with AddressOfEntryPoint, MajorLinkerVersion, and TimeDateStamp leading. The long tail of features with small but non-zero importance values suggests that many of the additional features contribute incrementally to classification decisions, collectively accounting for the observed accuracy improvement.

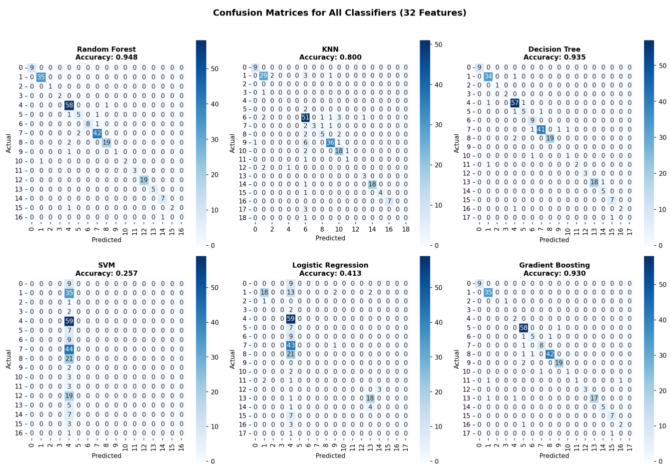


Fig. 10: Confusion Matrices - Extended Feature Set (32 Features)

Figure 10 presents the confusion matrices for the extended feature set. Compared to Figure 5, the diagonal entries are more prominent for Random Forest, Decision Tree, and Gradient Boosting, indicating fewer misclassifications. KNN's confusion matrix also shows notable improvement with more concentrated diagonal predictions, while SVM and Logistic Regression remain largely unchanged.

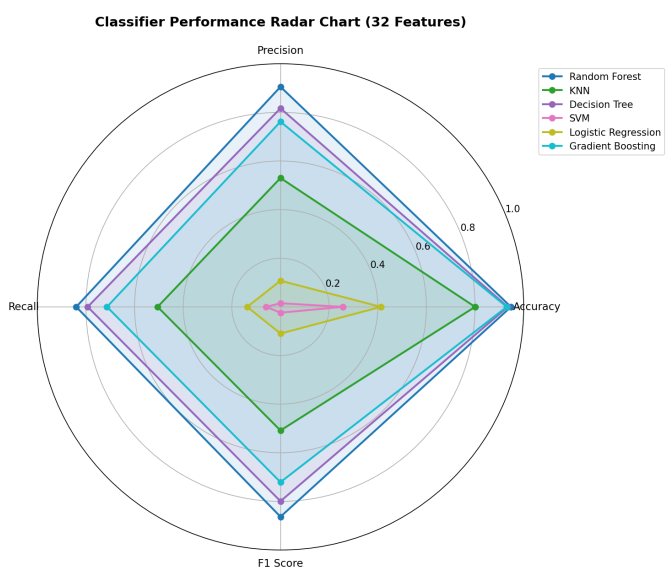


Fig. 11: Performance Radar Chart - Extended Feature Set (32 Features)

Figure 11 shows the radar chart for the extended feature set. Random Forest's coverage area is the largest and most symmetric, reflecting balanced high performance. The expansion of KNN's shape compared to Figure 6 visually confirms its improvement with additional features, while SVM and Logistic Regression shapes remain near the center.

VI. Conclusion

This research presents a comprehensive comparative study of PE header-based feature extraction for static malware analysis. The proposed methodology achieves a 75% reduction in feature extraction time compared to prior work, processing 1,150 malware samples in only 215–290 seconds depending on feature set size, compared to the 19-minute extraction time reported for the APT1 dataset. This improvement enables more practical deployment in real-time malware detection scenarios.

The extended feature set (32 features) outperforms the reduced feature set (14 features) for tree-based and instance-based classifiers. Random Forest achieved the highest accuracy at 94.78% with the extended feature set, while tree-based methods and KNN improved accuracy by 0.87% to 4.78%. However, Logistic Regression performed worse with the extended set (41.30% vs. 51.30%), likely due to multicollinearity among the additional features, and SVM exhibited poor performance overall due to the multi-class nature of the problem and the lack of feature scaling. While the extended feature set provides improved accuracy, it requires approximately 34% more extraction time (290 seconds vs. 215 seconds). For applications prioritizing speed over marginal accuracy improvements, the reduced feature set offers an effective compromise.

The research demonstrates that efficient malware detection is achievable using only PE header features without requiring computationally expensive dynamic analysis. The methodology is suitable for real-time malware scanning applications, large-scale malware dataset analysis, integration with existing antivirus systems, and automated malware triage systems.

Several limitations should be acknowledged. The dataset comprises 1,150 samples from a single source (MalwareBazaar), which may not fully represent the diversity of real-world malware and potentially introduces selection bias. The dataset also lacks benign samples for binary classification evaluation. Static analysis cannot detect malware that employs runtime obfuscation or packing techniques, and PE header features alone may not capture behavioral characteristics of sophisticated malware. The methodology is specific to Windows PE executables and does not apply to other platforms. Additionally, SVM and Logistic Regression were not optimized with feature scaling or extensive hyperparameter tuning, and cross-validation was limited to 3 folds due to computational constraints.

VII. Future Work

Future work should expand the dataset to include 10,000+ samples with benign files and multiple malware families, explore feature selection techniques (PCA, t-SNE) to identify optimal feature subsets, implement parallel feature extraction using multi-core processors or distributed computing, develop ensemble models combining multiple classifiers for improved robustness, explore deep learning approaches (CNN, LSTM) for automatic feature learning [8], and evaluate the methodology in a production-ready real-time malware detection system integrated with existing security infrastructure.

Acknowledgements

We thank MalwareBazaar (abuse.ch) for providing access to malware samples.

Generative AI Disclosure: This paper was partially drafted and revised with the assistance of Claude (Anthropic). Specifically, Claude was used to restructure and rewrite portions of the abstract, introduction, background, motivation, working mechanism, results discussion, and conclusion sections based on the authors' original content and detailed instructions. The AI tool was also used to format the paper in SCITEPRESS LaTeX style. All AI-generated content was manually reviewed, verified, and approved by the authors. Experimental results, data, feature extraction code, and classifier outputs were produced entirely by the authors and validated independently by re-running the code and crosschecking against the reported tables and figures.

References

[1] R. Kalakuntla, A. B. Vanamala, and R. R. Kolipyaka, "Cyber security," *HOLISTICA-Journal of Business and Public Administration*, vol. 10, no. 2, pp. 115–128, 2019.

[2] P. Black and J. Opacki, "Anti-analysis trends in banking malware," in *2016 11th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 1–7, IEEE, 2016.

[3] N. Balram, G. Hsieh, and C. McFall, "Static malware analysis using machine learning algorithms on APT1 dataset with string and PE header features," in *2019 International Conference on Computational Science and Computational Intelligence (CSCI)*, pp. 90–95, IEEE, 2019.

[4] T. Rezaei and A. Hamze, "An efficient approach for malware detection using PE header specifications," in *2020 6th International Conference on Web Research (ICWR)*, pp. 234–239, IEEE, 2020.

[5] R. S. Santos and E. D. Festijo, "Generating features of windows portable executable files for static analysis using portable executable reader module (pefile)," in *2021 4th International Conference of Computer and Informatics Engineering (IC2IE)*, pp. 283–288, IEEE, 2021.

[6] N. Maleki and H. Rastegari, "An improved method for packed malware detection using PE header and section table information," *International Journal of Computer Network & Information Security*, vol. 11, no. 9, 2019.

[7] H. H. Al-Khshali, M. Ilyas, and O. N. Ucan, "Effect of PE file header features on accuracy," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1115–1120, IEEE, 2020.

[8] S. Kumar *et al.*, "MCFT-CNN: Malware classification with fine-tune convolution neural networks using traditional and transfer learning in internet of things," *Future Generation Computer Systems*, vol. 125, pp. 334–351, 2021.

[9] C. K. Yuk and C. J. Seo, "Static analysis and machine learning-based malware detection system using PE header feature values," *International Journal of Innovative Research and Scientific Studies*, vol. 5, no. 4, pp. 281–288, 2022.

[10] K. K. R. Choo and A. Dehghantanha, *Handbook of Big Data Analytics and Forensics*. Springer, 2022.

[11] abuse.ch, "MalwareBazaar: Malware sample exchange," Available: https://malwarebazaar.abuse.ch, Accessed: 2024.